



## Università degli Studi di Padova

Laurea: Informatica

Corso: Ingegneria del Software

Anno Accademico: 2025/26



## Gruppo 17

Nome: BitByBit

Email: swe.bitbybit@gmail.com

## Glossario

|                      |                                                                               |
|----------------------|-------------------------------------------------------------------------------|
| <b>Stato:</b>        | Completato                                                                    |
| <b>Versione:</b>     | 2.0.0                                                                         |
| <b>Redattori:</b>    | Ferdinando Fracasso<br>Riccardo Manisi<br>Giovanni Visentin<br>Dennis Parolin |
| <b>Verificatori:</b> | Gabriele Scaggianti<br>Dennis Parolin<br>Giovanni Visentin                    |
| <b>Responsabile:</b> | Giovanni Visentin                                                             |
| <b>Destinatari:</b>  | Gruppo BitByBit                                                               |

## Registro delle modifiche

| Versione | Data       | Autore              | Descrizione                                                                                                                                                               | Verificatore        |
|----------|------------|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|
| 1.0.0    | 2026-03-02 | Giovanni Visentin   | Documento in versione stabilizzata                                                                                                                                        | Dennis Parolin      |
| 0.4.0    | 2026-02-27 | Marco Sanguin       | Aggiunti Indice Analitico                                                                                                                                                 | Riccardo Manisi     |
| 0.3.2    | 2025-12-23 | Riccardo Manisi     | Aggiunta sezione info generali documenti e migliorato layout                                                                                                              | Dennis Parolin      |
| 0.3.1    | 2025-12-16 | Ferdinando Fracasso | Rimosso definizioni di termini non utilizzati e tolta la sezione introduttiva perchè superflua, inoltre modificato definizioni di branch/merge/pull request per chiarezza | Giovanni Visentin   |
| 0.3.0    | 2025-12-12 | Ferdinando Fracasso | Ristrutturato layout documento e aggiunto sezione introduttiva                                                                                                            | Giovanni Visentin   |
| 0.2.1    | 2025-11-05 | Dennis Parolin      | Aggiunti termini. Fix errori generici                                                                                                                                     | Gabriele Scaggiante |
| 0.2.0    | 2025-11-06 | Riccardo Manisi     | Tolta la sezione delle convenzioni di scrittura, aggiunto indice e formattazione, inserito nuovi termini                                                                  | Dennis Parolin      |
| 0.1.0    | 2025-11-04 | Giovanni Visentin   | Stesura iniziale del glossario                                                                                                                                            | Gabriele Scaggiante |

## Indice

|                                     |    |
|-------------------------------------|----|
| <a href="#">Scopo del documento</a> | 7  |
| <a href="#">A</a>                   | 7  |
| <a href="#">B</a>                   | 9  |
| <a href="#">C</a>                   | 11 |
| <a href="#">D</a>                   | 13 |
| <a href="#">E</a>                   | 14 |
| <a href="#">F</a>                   | 14 |
| <a href="#">G</a>                   | 14 |
| <a href="#">H</a>                   | 15 |
| <a href="#">I</a>                   | 15 |
| <a href="#">J</a>                   | 16 |
| <a href="#">K</a>                   | 16 |
| <a href="#">L</a>                   | 16 |
| <a href="#">M</a>                   | 17 |
| <a href="#">N</a>                   | 19 |
| <a href="#">O</a>                   | 19 |
| <a href="#">P</a>                   | 19 |
| <a href="#">Q</a>                   | 22 |
| <a href="#">R</a>                   | 23 |
| <a href="#">S</a>                   | 25 |
| <a href="#">T</a>                   | 28 |
| <a href="#">U</a>                   | 29 |
| <a href="#">V</a>                   | 30 |

|          |           |
|----------|-----------|
| <b>W</b> | <b>31</b> |
| <b>X</b> | <b>32</b> |
| <b>Y</b> | <b>32</b> |
| <b>Z</b> | <b>32</b> |

# Indice analitico

Agile, [7](#)  
Analisi dei requisiti, [7](#)  
Analisi Dinamica, [7](#)  
Analisi Statica, [7](#)  
API REST, [8](#)  
Architettura, [8](#)  
Architettura a Strati, [8](#)  
  
Backend, [9](#)  
Baseline, [9](#)  
Black-box, [9](#)  
Board di progetto, [9](#)  
Boundary Analysis, [9](#)  
Branch, [9](#)  
Bugfix, [9](#)  
Business Logic, [10](#)  
  
Capitolato, [11](#)  
Checklist, [11](#)  
CI/CD, [11](#)  
Ciclo di vita, [11](#)  
Cloud, [11](#)  
Code Base, [11](#)  
Committente, [12](#)  
Configuration Item, [12](#)  
Continuous Verification, [12](#)  
Contratto, [12](#)  
  
Data Access Layer, [13](#)  
Data Layer, [13](#)  
Deployment, [13](#)  
Driver, [13](#)  
  
Event-Driven, [14](#)  
  
Framework, [14](#)  
  
Git Flow, [14](#)  
GitHub Actions, [14](#)  
  
IDE, [15](#)  
Infrastructure as Code, [15](#)  
Ispezione, [15](#)  
Issue, [15](#)  
  
lowerCamelCase, [16](#)  
  
Matrice di Tracciamento, [17](#)  
Merge, [17](#)  
Message Bus, [17](#)  
Metrica, [17](#)  
Milestone, [17](#)  
Model-View-ViewModel (MVVM), [17](#)  
MVP, [18](#)  
  
Payload, [19](#)  
PB, [19](#)  
Persistence Logic, [19](#)  
Piano di progetto, [19](#)  
Piano di qualifica, [20](#)  
PoC, [20](#)  
Presentation Logic, [20](#)  
Processo, [20](#)  
Product backlog, [20](#)  
Project, [21](#)  
Proponente, [20](#)  
Pull Request, [21](#)  
  
Quality Gate, [22](#)  
  
Refactoring, [23](#)  
Regressione, [23](#)  
Release, [23](#)  
Repository, [23](#)  
Requisito, [23](#)  
Requisito funzionale, [23](#)  
Requisito non funzionale, [23](#)  
Requisito software, [23](#)  
Requisito utente, [23](#)

RTB, [24](#)

Scalabilità dinamica, [25](#)

Scenari d'uso, [25](#)

Self-documenting code, [25](#)

SemVer, [25](#)

Separation of Concerns, [25](#)

Serverless, [25](#)

Single Point of Entry, [25](#)

Specifica Tecnica, [26](#)

Sprint, [26](#)

Sprint backlog, [26](#)

Sprint Retrospective, [26](#)

Sprint Review, [26](#)

Stakeholder, [26](#)

State Management, [26](#)

Statement Coverage, [26](#)

Stub, [27](#)

Test di Accettazione, [28](#)

Test di Integrazione, [28](#)

Test di Regressione, [28](#)

Test di Sistema, [28](#)

Test di Unità, [28](#)

To-Do, [28](#)

UI Layer, [29](#)

UpperCamelCase, [29](#)

Utente, [29](#)

Validazione, [30](#)

Verbale, [30](#)

Verifica, [30](#)

Walkthrough, [31](#)

Way of working, [31](#)

White-box, [31](#)

YAML, [32](#)

Zero Ops, [32](#)

## Scopo del documento

Il presente documento ha lo scopo di raccogliere i termini tecnici utilizzati all'interno del progetto e fornire le loro definizioni, in modo da permettere una più facile comprensione della documentazione prodotta durante lo svolgimento del progetto.

## A

### Agile

Metodo per lo sviluppo sw ideato per produrre sw utile nel minor tempo possibile. Il sistema è sviluppato attraverso una serie di incrementi a cui partecipano anche gli stakeholders e gli utenti finali per una maggiore comunicazione in fase di sviluppo. I risultati di ogni incremento vengono utilizzati come base per la pianificazione del prossimo. Attività centrali sono: il **design** e l'**implementazione**. La stesura di un documento dei requisiti è superflua in quanto i requisiti cambiano così velocemente che il documento è obsoleto non appena viene scritto.

### Analisi dei requisiti

Stabilisce esattamente cosa deve essere implementato nel sistema, cioè elenca i requisiti lato soluzione (requisiti software). Può essere parte del contatto con l'acquirente. Può essere soggetto a modifiche durante lo sviluppo del prodotto sw.

### Analisi Dinamica

Tecnica di verifica che richiede l'esecuzione del codice software su un processore (reale o virtuale).

- Il suo obiettivo è osservare il comportamento del sistema in risposta a determinati input per rilevare malfunzionamenti (failure).
- Include le attività di testing (Unità, Integrazione, Sistema).

### Analisi Statica

Tecnica di verifica che non prevede l'esecuzione del codice.

- Si basa sull'esame dei documenti o del codice sorgente per individuare difetti di forma, sintassi o logica.
- Include attività come l'ispezione, il walkthrough e l'uso di strumenti automatici (linter).

## **API REST**

Acronimo di *Representational State Transfer*. Stile architetturale per sistemi distribuiti che definisce un set di vincoli e principi per la creazione di interfacce web, basato sullo scambio logico di messaggi senza stato tramite il protocollo HTTP.

## **Architettura**

Organizzazione generale del sistema e dei suoi componenti.

## **Architettura a Strati**

Modello architetturale (noto come *Layered Architecture*) in cui i componenti del software sono disposti in livelli gerarchici isolati. Ogni strato ha una responsabilità specifica e comunica solitamente solo con gli strati adiacenti, favorendo la separazione degli intenti e la manutenibilità.



## B

### Baseline

Versione approvata di un prodotto di lavoro (di progetto) che può essere modificato solo attraverso procedure formali di controllo delle modifiche.

### Backend

Parte del sistema software non direttamente visibile o accessibile all'utente finale, responsabile dell'elaborazione della logica di business, dell'accesso ai database e della gestione dell'infrastruttura di calcolo.

### Black-box

Metodologia di test in cui il verificatore non conosce o ignora la struttura interna del sistema (codice sorgente).

- I test sono progettati basandosi esclusivamente sulle specifiche dei requisiti (input e output attesi).
- È nota anche come testing funzionale.

### Board di progetto

Rappresenta un'organizzazione delle attività, nel contesto di GitHub identificate tramite le issue. Può contenere tutte le attività di un progetto o solo un sottoinsieme con caratteristiche precise.

### Boundary Analysis

Tecnica di progettazione dei test black-box che si concentra sui valori ai margini delle classi di equivalenza degli input (es. minimo, massimo, appena sopra, appena sotto). È utilizzata perché i difetti si nascondono spesso ai bordi dei range di input validi.

### Branch

Versione parallela del repository. È contenuto all'interno del repository, ma non ha effetti sul branch principale. Permette di lavorare senza interrompere la versione stabile corrente.

### Bugfix

Intervento di manutenzione correttiva volto a risolvere un difetto (bug) riscontrato nel software. Include la diagnosi del problema, la modifica del codice e la successiva verifica.

## **Business Logic**

Insieme delle regole, dei calcoli e dei processi che definiscono il comportamento funzionale di un'applicazione, indipendentemente dalla presentazione visiva e dall'accesso ai dati. Nel contesto dell'architettura Flutter adottata dal gruppo, la Business Logic risiede nel Data Layer, all'interno delle classi **Service**.

## C

### Capitolato

Documento che contiene le specifiche di un progetto software. Viene redatto dal proponente e presentato ai fornitori interessati a partecipare all'appalto. Presenta aspettative, vincoli, e suggerimenti. Descrive i requisiti lato bisogno (requisiti utente).

### Checklist

Lista di controllo strutturata utilizzata durante le ispezioni per guidare il verificatore. Contiene un elenco di criteri ed errori comuni da cercare nell'artefatto esaminato, garantendo che nessun aspetto critico venga trascurato.

### CI/CD

Insieme di pratiche e strumenti che automatizzano le fasi di integrazione, test e rilascio del software.

- **CI:** (Continuous Integration) integra frequentemente le modifiche di codice in un repository condiviso, eseguendo build e test automatici.
- **CD:** (Continuous Delivery) automatizza il rilascio del software negli ambienti di test o produzione.

### Ciclo di vita

È composto da un insieme di stati che descrivono l'avanzamento del prodotto software dalla concezione all'utilizzo fino al ritiro. Si riferisce alla completa durata dello sviluppo del prodotto, dal concepimento alla fine, che deve essere garantita. È composto da stati in cui si ha una certa sequenza di passaggi da seguire. La transizione da uno stato all'altro avviene come conseguenza di un processo di ciclo di vita. In seguito al ritiro il prodotto può essere rimesso in vita, per questo viene indicato con ciclo.

### Cloud

Modello di erogazione di risorse informatiche (server, storage, database, calcolo) su richiesta tramite Internet, che svincola l'acquirente dalla necessità di possedere e mantenere hardware fisico in loco.

### Code Base

Insieme completo del codice sorgente che costituisce un'applicazione o un sistema software, tipicamente gestito all'interno di un repository di controllo di versione.

## **Committente**

Colui che ordina l'esecuzione di un prodotto ad un ente esterno, il fornitore. È colui che ha il maggior peso contrattuale.

## **Configuration Item**

Identifica una componente o un elemento infrastrutturale che deve essere gestito e monitorato per garantire la corretta erogazione dei servizi software.

## **Continuous Verification**

Approccio che integra la verifica (statica e dinamica) in ogni fase del flusso di sviluppo, anziché posticiparla alla fine. Nel contesto del gruppo, è implementata tramite policy che impediscono il merge di codice non verificato.

## **Contratto**

Documento (legale) stilato tra committente e fornitore.

## D

### **Data Access Layer**

Acronimo DAL. Strato architetturale che semplifica e uniforma l'accesso ai dati persistenti. Espone operazioni fondamentali ai layer superiori celando loro l'implementazione fisica e i dettagli del database sottostante.

### **Data Layer**

Strato architetturale responsabile della gestione e dell'accesso ai dati dell'applicazione. Nel contesto dell'architettura Flutter adottata dal gruppo, è composto dalle classi **Repository**, **Service** e **Model**, ed è l'unico strato autorizzato a comunicare con sorgenti dati esterne.

### **Deployment**

Il processo di messa in opera di un sistema software o di un aggiornamento in un ambiente specifico (es. ambiente di staging o di produzione), rendendolo disponibile per l'uso o per il collaudo.

### **Driver**

Componente software fittizio utilizzato nei test di integrazione (strategia Bottom-Up). Simula il comportamento di un modulo di livello superiore che deve chiamare e pilotare il modulo sotto test, passandogli gli input necessari.

## **E**

### **Event-Driven**

Paradigma architetturale in cui il flusso del programma, i servizi e lo scambio di dati sono guidati dalla produzione e dal rilevamento di eventi asincroni. Favorisce una fortissima riduzione dell'accoppiamento tra i componenti applicativi.

## **F**

### **Framework**

Architettura logica di supporto su cui il software viene strutturato. Fornisce funzionalità predefinite e detta precise regole implementative per velocizzare lo sviluppo ed estendere la standardizzazione del codice.

## **G**

### **Git Flow**

Modello di gestione dei branch in Git che definisce ruoli specifici per i diversi rami (es. `main`, `develop`, `feature/*`) e regole precise per la loro interazione, facilitando il lavoro parallelo e il rilascio di versioni.

### **GitHub Actions**

Piattaforma di integrazione continua (CI) e distribuzione continua (CD) integrata in GitHub, che permette di automatizzare i flussi di lavoro di compilazione, test e deployment direttamente dal repository.

## H

## I

### IDE

Applicazione software che fornisce agli sviluppatori strumenti completi per lo sviluppo software. Include solitamente un editor di codice, strumenti di build automation, un debugger e funzionalità di intellisense (es. Visual Studio Code).

### Infrastructure as Code

Prassi che consiste nel tracciamento e nel provisioning di data center e infrastrutture in cloud tramite file di testo leggibili dalla macchina (detti *template*), anziché mediante l'uso di interfacce grafiche o configurazioni manuali.

### Ispezione

Tecnica formale di analisi statica in cui un verificatore esamina un artefatto (codice o documento) in autonomia, utilizzando una checklist per individuare discrepanze rispetto agli standard. È un processo rigoroso e documentato.

### Issue

Unità di lavoro tracciata su un sistema di gestione progetti (es. GitHub). Nel contesto della validazione, viene utilizzata per tracciare anomalie o bug riscontrati durante i test.

## **J**

## **K**

## **L**

### **lowerCamelCase**

Convenzione di nomenclatura in cui la prima parola inizia con lettera minuscola e ogni parola successiva inizia con lettera maiuscola, senza separatori (es. `userName`, `fetchData`). In Dart è utilizzata per variabili, parametri, metodi e costanti.



## M

### **Matrice di Tracciamento**

Tabella che mappa le relazioni tra i requisiti e altri artefatti del progetto, in particolare i test. Serve a garantire che ogni requisito abbia almeno un test associato (copertura) e che ogni test sia giustificato da un requisito.

### **Milestone**

Strumento utilizzato nella gestione di progetto per fissare una data di calendario che descrive degli obiettivi che portano un avanzamento nello sviluppo di progetto. Ogni milestone deve essere:

- specifica
- raggiungibile
- misurabile
- traducibile in compiti assegnabili
- dimostrabile dagli stakeholders

### **Merge**

Metodo per integrare i cambiamenti fatti in un branch di lavoro in un altro branch, definito di destinazione. Può essere eseguito con le pull request o tramite la command line.

### **Message Bus**

Infrastruttura middleware di comunicazione che consente il passaggio di eventi tra componenti in modo disaccoppiato. Funge da intermediario che riceve i messaggi e provvede ad inviarli ai servizi destinatari, garantendone l'inoltro e la resilienza agli errori.

### **Metrica**

Qualsiasi tipo di misurazione relativa a un sistema software, processo o documentazione. Permette di quantificare un attributi di un prodotto o un processo.

### **Model-View-ViewModel (MVVM)**

Pattern architetturale ideato per separare nettamente lo sviluppo dell'interfaccia utente grafica (View) dalla logica di business e dai dati (Model). Tale separazione avviene per mezzo del ViewModel, che funge da intermediario reattivo per la conversione dei dati.

## MVP

Sigla per **Minimum Viable Product**. Artefatto che definisce la versione di un prodotto dotata di funzionalità sufficienti per essere distribuito efficacemente. Lo scopo è quello di ottenere dei feedback da parte del committente e/o dei potenziali utenti utilizzatori. La creazione di questo prodotto deve consumare la minima quantità di risorse del fornitore, in quanto può essere scartato completamente.

## N

## O

## P

### Payload

La porzione di dati utili e significativi trasportata all'interno di un messaggio scambiato in rete (come una richiesta HTTP), escludendo i metadati o le intestazioni tecniche (*headers*) necessarie per formattare il pacchetto.

### PB

Sigla per **Product Baseline**. Baseline di progetto per validare l'architettura del prodotto software e la sua realizzazione. Espone il design definitivo del prodotto descritto nel documento delle specifiche tecniche.

### Persistence Logic

Insieme delle operazioni responsabili del salvataggio, del recupero e della trasformazione dei dati verso e da una sorgente dati persistente (es. database, API REST). Nel contesto dell'architettura Flutter adottata dal gruppo, la Persistence Logic risiede nel Data Layer, all'interno delle classi **Repository**.

### Piano di progetto

Documento ufficiale di tipo esterno, soggetto ad approvazioni, con il quale si descrivono gli obiettivi di progetto e gli elementi necessari al loro raggiungimento. Si vedono le risorse disponibili e le loro assegnazioni alle attività con scansione nel tempo. Principalmente è scomposto in:

- Pianificazione (preventiva): Team, Analisi dei rischi
- Consuntivazione: valutazione retrospettiva delle attività svolte

Struttura tipica del PdP:

- Introduzione (scopo e struttura)
- Organizzazione del progetto
- Analisi dei rischi

- Risorse disponibili
- Suddivisione del lavoro (work breakdown)
- Calendario delle attività
- Meccanismo di controllo e rendicontazione

## Piano di qualifica

Documento per la specifica degli obiettivi di qualità. Espone gli standard adottati per la misurazione della qualità e le componenti che devono essere oggetto di test di verifica. Vengono presentate anche le misurazioni effettuate tramite il cruscotto della qualità durante gli sprint di progetto.

## PoC

Sigla per **Proof of Concept**. Artefatto che serve a valutare la fattibilità tecnologica del prodotto che si intende sviluppare. Le tecnologie adottate devono essere decise in base alle specifiche funzionalità individuate a partire dai bisogni del committente.

## Presentation Logic

Insieme delle operazioni responsabili della rappresentazione grafica degli oggetti di dominio dell'applicazione. Nel contesto dell'architettura Flutter adottata dal gruppo, la Presentation Logic risiede nel Data Layer, all'interno delle classi `ViewModel`.

## Processo

Un insieme strutturato di attività, metodi, pratiche e trasformazioni utilizzate per sviluppare, mantenere e gestire un prodotto software o un sistema. Definisce cosa deve essere fatto, chi lo fa, quando e come, per raggiungere un obiettivo specifico.

## Product backlog

Lista dinamica che evolve nel tempo con il prodotto. Vengono rappresentati gli item da eseguire nei prossimi rilasci con un livello di priorità assegnato a ciascuno. Gli item presenti rappresentano le macroattività che devono essere eseguite dal team.

## Proponente

Azienda esterna all'università che propone un capitolato da sviluppare.

## **Project**

È un insieme ordinato di attività sviluppate sulla base dei processi di ciclo di vita che soddisfano gli obiettivi dati.

- Le attività sono suddivise in compiti assegnabili a singoli individui.
- Le attività sono pianificate prima di essere svolte.

## **Pull Request**

Modifiche proposte da un repository inviate da un utente e accettate o scartate da un collaboratore interno al repository.

## Q

### **Quality Gate**

Punto di controllo nel ciclo di vita del software che il progetto deve superare per passare alla fase successiva. Nel contesto della CI/CD, è un insieme di criteri automatici (es. "nessun test fallito") che bloccano il merge se non soddisfatti.

## **R**

### **Refactoring**

Processo di ristrutturazione del codice esistente senza modificarne il comportamento esterno. L'obiettivo è migliorare la leggibilità, ridurre la complessità e facilitare la manutenzione futura.

### **Regressione**

Malfunzionamento introdotto involontariamente in una funzionalità precedentemente funzionante a seguito di una modifica al codice (es. bugfix o nuova feature).

### **Release**

Una versione particolare di un configuration item resa disponibile per uno scopo specifico.

### **Repository**

Archivio centralizzato dove viene conservato e gestito il codice sorgente del progetto.

### **Requisito**

Condizione o capacità che il sistema deve soddisfare.

### **Requisito funzionale**

Funzionalità che il sistema deve offrire.

### **Requisito non funzionale**

Vincolo di qualità (prestazioni, sicurezza, usabilità, manutentibilità).

### **Requisito software**

Descrizioni dettagliate delle funzioni, servizi e vincoli di sistema, inseriti nel documento di analisi dei requisiti.

### **Requisito utente**

Requisiti astratti di alto livello. Dichiarazioni in linguaggio naturale (e anche diagrammi) dei servizi che il sistema dovrebbe fornire agli utenti e dei vincoli che deve rispettare.

## **RTB**

Sigla per **Requirements and Technology Baseline**. Rappresenta una baseline di progetto. Produce la lista dei requisiti funzionali e non funzionali da soddisfare, stabiliti con la proponente e il PoC.



## S

### **Scalabilità dinamica**

Capacità di un sistema cloud di adattare, allocare o liberare automaticamente le risorse computazionali in tempo reale, allo scopo di seguire attivamente le variazioni imprevedibili del traffico applicativo.

### **Scenari d'uso**

Descrizioni narrative di come un utente interagisce con il sistema per raggiungere un obiettivo specifico. Sono utilizzati durante la validazione per verificare che il flusso di lavoro supporti le esigenze reali.

### **Self-documenting code**

Codice scritto in modo talmente chiaro (tramite nomi significativi di variabili e funzioni) da rendere superflui i commenti esplicativi per comprenderne il funzionamento logico.

### **SemVer**

Sigla per **Semantic Versioning**. Metodologia composta da un insieme di regole e vincoli che specifica come i numeri di versione sono incrementati e assegnati.

### **Separation of Concerns**

Principio di progettazione informatica in cui un programma è diviso in sezioni o ambiti logicamente distinti. Ciascuna sezione è responsabile di un'unica e sola area di interesse senza sovrapposizioni logiche con le altre.

### **Serverless**

Modello di esecuzione o paradigma afferente al cloud computing in cui il fornitore infrastrutturale gestisce dinamicamente l'assegnazione e lo spegnimento dei server. Il consumatore del servizio provvede esclusivamente allo sviluppo della logica di calcolo.

### **Single Point of Entry**

Nodo applicativo o gateway infrastrutturale concepito per essere l'unica interfaccia di accesso che il sistema espone per accettare input esterni. Tale limitazione facilita la centralizzazione del traffico, delle validazioni e dei controlli di sicurezza.

## **Specifica Tecnica**

Documento che descrive dettagliatamente le scelte fatte in ambito di progettazione e implementazione, spiegando il motivo di tali scelte e fornendo informazioni tecniche necessarie per comprendere il sistema.

## **Sprint**

Breve periodo di tempo, dalla durata prefissata, in cui un team lavora per completare le attività elencate nello sprint backlog di quel preciso sprint.

## **Sprint backlog**

Lista degli item riferita ad un particolare sprint di progetto. Gli item al suo interno possono essere scomposti in attività più piccole, ma non possono essere mai eliminati.

## **Sprint Retrospective**

Riunione che viene fatta con la sola presenza dei membri del team per identificare le attività portate a termine e i problemi insorti durante la loro esecuzione. Ha l'obiettivo di identificare i potenziali errori fatti.

## **Sprint Review**

Riunione, fatta alla fine di uno sprint, che consta di una revisione per identificare il progresso eseguito sul prodotto. Partecipano anche gli stakeholders.

## **Stakeholder**

Tutti coloro che a vario titolo hanno influenza sul prodotto e sul progetto: utenti, committente, fornitore, regolatori.

## **State Management**

Nello sviluppo frontend, identifica l'approccio centralizzato per tracciare e gestire le variabili di stato reattivo di un'applicazione. Il suo scopo è garantire che qualsiasi interazione dell'utente modifichi i dati globali o locali e li rifletta automaticamente nel render visivo.

## **Statement Coverage**

Metrica di copertura del codice che indica la percentuale di istruzioni (righe di codice) eseguite almeno una volta durante i test automatici.

## **Stub**

Componente software fittizio utilizzato nei test di integrazione (strategia Top-Down). Simula il comportamento di un modulo di livello inferiore non ancora implementato, fornendo risposte predefinite alle chiamate del modulo sotto test.

## T

### Test di Accettazione

Livello di test finalizzato a validare il sistema rispetto ai bisogni dell'utente.

- Viene eseguito in un ambiente simile a quello reale.
- Ha lo scopo di ottenere l'approvazione formale del cliente per il rilascio (Validazione).

### Test di Integrazione

Livello di test che verifica le interazioni tra componenti software o sistemi diversi. L'obiettivo è rilevare difetti nelle interfacce e nello scambio di dati tra moduli che singolarmente funzionano.

### Test di Regressione

Riesecuzione di test già superati in precedenza per assicurarsi che le recenti modifiche al codice non abbiano introdotto nuovi difetti in parti del software già verificate.

### Test di Sistema

Livello di test che verifica il comportamento dell'intero sistema integrato rispetto ai requisiti specificati. Copre aspetti funzionali e non funzionali (prestazioni, sicurezza).

### Test di Unità

Livello di test che verifica le più piccole parti testabili del software (es. singole funzioni o metodi) in isolamento dal resto del sistema.

### To-Do

Attività che derivano da decisioni prese dai componenti del gruppo. Ogni To-Do deve essere tracciabile all'interno del contesto del progetto tramite la sua formalizzazione in issue GitHub.

## U

### UI Layer

Strato architetturale responsabile della presentazione dei dati all'utente e della gestione degli input. Nel contesto dell'architettura Flutter adottata dal gruppo, è composto dalle classi `Screen`, `ViewModel` e `Widget`, e non deve contenere logica di business o accesso diretto ai dati.

### UpperCamelCase

Convenzione di nomenclatura in cui ogni parola, inclusa la prima, inizia con lettera maiuscola, senza separatori (es. `UserRepository`, `AuthService`). In Dart è utilizzata per classi, enum, typedef ed estensioni.

### Utente

La persona che usa e interagisce direttamente col sistema (spesso si identifica col commit-tente).

## V

### **Validazione**

Processo di conferma, attraverso evidenze oggettive, che il software soddisfi le esigenze dell'utente e l'uso previsto. Risponde alla domanda: "Abbiamo costruito il prodotto giusto?".

### **Verbale**

Documento riassuntivo di un incontro. Deve specificare Ordine del Giorno, Discussioni, Decisioni, Issue e To Do. Ogni verbale deve essere verificato da un membro diverso da chi l'ha scritto.

### **Verifica**

Processo di valutazione per determinare se i prodotti di una fase di sviluppo soddisfano le condizioni imposte all'inizio di quella fase. Risponde alla domanda: "Stiamo costruendo il prodotto nel modo giusto?".

## W

### **Walkthrough**

Tecnica di analisi statica informale in cui l'autore di un artefatto guida i membri del team attraverso il documento o il codice per individuare errori e condividere conoscenza.

### **Way of working**

Comprende le metodologie di sviluppo (modello di sviluppo), la spartizione dei ruoli e delle attività, gli strumenti di lavoro (GitHub, GitLab, Trello, Jira, Slack, Discord, Obsidian, Google Docs), convenzioni standard e procedure condivise dai membri del team.

### **White-box**

Metodologia di test in cui il verificatore conosce la struttura interna del codice e la utilizza per progettare i casi di test.

- Mira a coprire i percorsi logici interni (es. rami if-else, cicli).
- È nota anche come testing strutturale.

## X

## Y

### YAML

Acronimo ricorsivo per *YAML Ain't Markup Language*. Linguaggio di rappresentazione per la serializzazione dei dati, leggibile universalmente dall'uomo e largamente utilizzato per la stesura di file di configurazione per logiche CI/CD o per infrastrutture.

## Z

### Zero Ops

Acronimo di *Zero Operations*. Paradigma aziendale e logico in cui i team di sviluppo responsabili del prodotto sono sollevati da tutte le operazioni manuali di manutenzione, aggiornamento o allocazione dei servizi fisici che ospitano il sistema.